parameter `id` to the endpoint parameter. The endpoint parameter type `ProductId` implements `TryParse`. If parsing is successful, the parsed parameter is used to call the endpoint handler. If parsing fails, the endpoint middleware returns a `400 Bad Request` response.

Listing 7.2 shows how you could implement the `TryParse` method for `ProductId`. This method creates a `ProductId` from `string`s that consist of an `integer` prefixed with `'p'` (`p123` or `p456`, for example). If the input `string` matches the required format, it creates a `ProductId` instance and returns `true`. If the format is invalid, it returns `false`, binding fails, and a `400 Bad Request` is returned.

Listing 7.2 Implementing `TryParse` in a custom type to allow parsing from route values

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

app.MapGet("/product/{id}", (ProductId id) => $"Received {id}");     ❶
app.Run();

readonly record struct ProductId(int Id)                             ❷
{
    public static bool TryParse(string? s, out ProductId result)    ❸
    {
        if(s is not null                                             ❹
           && s.StartsWith('p')                                      ❹
           && int.TryParse(                                          ❺
                s.AsSpan().Slice(1),                                 ❻
                out int id))                                         ❼
        {
            result = new ProductId(id);                              ❽
            return true;                                             ❽
        }

        result = default;                                            ❾
        return false;                                                ❾
```